Phase 1 — Estimator Core: Baseline Report

Project: Visual-Inertial Odometry (VIO) Estimator **Phase:** 1 — Estimator Core **Status:** Complete (Steps 1–6 validated; Step 7 C++ port not yet started) **Date:** August 2026

---

1. Objective

Build and validate a working visual-inertial odometry estimator against ground truth, producing quantified baseline accuracy numbers to serve as the comparison point for later phases.

2. Summary

A monocular error-state EKF (ESKF) fusing IMU pre-integration with a sparse visual front end was implemented in Python, unit-tested against synthetic ground-truth data, and run end-to-end on a generated dataset with known-exact ground truth. The estimator holds reasonable accuracy over short windows (~seconds) and accumulates several meters of drift over a 20-second run. Diagnostic testing identified the specific, well-understood cause of this drift (Section 6) rather than leaving it as an unexplained result — this is a documented limitation of the Phase 1 landmark-update strategy, not a software defect.

3. Environment

Component	**Value**
Language	Python 3.14
Key libraries	NumPy, SciPy, OpenCV
Platform	Windows 11, PowerShell
Filtering approach	Hand-rolled error-state EKF (not `filterpy`)

4. Dataset

Real-dataset hosting proved unreliable during this phase: EuRoC MAV's primary host (`robotics.ethz.ch`) was unreachable across multiple networks (including a VPN and multiple WiFi networks), ETH's Research Collection mirror only offers a 12GB multi-sequence bundle (also failed to download), and TUM VI's individual sequence archives do not bundle per-sequence camera calibration files, which are published separately and were not reliably locatable.

Resolution: a synthetic dataset generator (`scripts/generate_synthetic_dataset.py`) was built instead, producing a circular flight trajectory with:

- Exactly known ground truth (position + orientation at every instant, analytically defined)
- Simulated IMU measurements (200 Hz) with realistic noise and constant bias, derived analytically from the trajectory
- Simulated camera images (20 Hz, 640×480) with ~300 trackable synthetic landmark features
- Output in the same EuRoC-format folder structure as `src/dataset.py` already expects, so no pipeline code needed to change

This has a real benefit beyond working around hosting problems: because ground truth is exact and noise is controlled, it enabled the root-cause diagnostics in Section 6 that would have been much harder to run conclusively against a real dataset.

Trajectory used for all results below: 20-second circular flight, radius 5m, base altitude 2m with ±0.3m sinusoidal bob, angular rate 0.3 rad/s (~1 full circle in ~21s), forward-facing camera, 400 image frames, 4,000 IMU samples.

5. Baseline Accuracy Results

5.1 Environment validation (unit tests)

Before any dataset run, all 5 core-math sanity checks passed (`tests/sanity_checks.py`):

Check	**Result**
Static IMU stays at rest	PASS
Constant acceleration matches Newtonian kinematics	PASS
Pre-integration matches step-by-step EKF propagation	PASS
ATE of identical trajectories is zero	PASS
Landmark update reduces pose error	PASS (0.374m → 0.192m)

5.2 Full pipeline run — primary result

Metric	Value
ATE RMSE	**3.4172 m**
ATE Mean	3.1678 m
ATE Median	3.1918 m
ATE Std	1.2816 m
ATE Max	6.3059 m
Recovered scale (monocular)	0.1172
RPE Trans RMSE (1-frame delta)	**0.3507 m**
RPE Rot RMSE (1-frame delta)	0.2977°

Context: over a 20m-diameter circular path, an ATE RMSE of ~3.4m indicates the trajectory *shape* is being tracked (confirmed visually — see Section 6.2) but with substantial accumulated drift, consistent with the root cause identified below.

5.3 Short-window observation (development testing)

An earlier 8-second run (shorter duration, same generator) produced a notably better ATE RMSE of approximately 1.36m — indicating error grows substantially with run duration, as expected for an estimator without a mechanism to bound long-term drift (Section 6).

6. Root-Cause Investigation

Rather than leave "3.4m of drift" as an unexplained number, three targeted diagnostic experiments were run to isolate its cause.

6.1 IMU-only propagation (no visual updates at all)

Running the EKF's `predict()` step alone against the full 20s of IMU data, with zero visual corrections:

Time	Estimated position	True position (approx.)
t=0.0s	[5.00, 0.00, 2.00]	[5.00, 0.00, 2.00]
t=2.5s	[3.75, -0.27, 1.99]	[3.66, 3.41, 2.28]

Time	Estimated position	True position (approx.)
t=10.0s	[-1.86, -12.7, 1.51]	diverging
t=17.5s	[12.74, -22.21, 3.31]	diverging

Filter covariance trace grew from 0.0015 at t=0 to over 6,200 by t=17.5s.

Conclusion: pure inertial dead-reckoning diverges to meters of error within seconds. **This is expected physics** — double-integrating noisy acceleration twice amplifies error quadratically over time — and confirms the IMU mechanization (`predict()`) is implemented correctly. It also establishes why visual corrections are necessary at all.

6.2 Landmark update correctness (using ground-truth poses)

To isolate whether the EKF's landmark reprojection update itself is mathematically sound, landmarks were triangulated using the *true* ground-truth camera poses (rather than the filter's own estimate) and applied as updates to the filter's (deliberately perturbed) state:

Frame	Pose error before update	Pose error after update
1	0.075 m	0.035 m
2	0.106 m	0.073 m
5	0.236 m	0.206 m
10	0.223 m	0.086 m
15	0.073 m	0.070 m

Every single update reduced pose error. This confirms the reprojection Jacobians and Kalman update logic in `ekf.py` are correct — given a good triangulation, the filter reliably moves toward the truth.

6.3 Triangulation baseline sensitivity

A hypothesis was tested: at 20Hz, consecutive camera frames are only ~7.5cm apart while landmarks are ~5m away — a baseline-to-depth ratio under 1° of parallax, which is a poorly-conditioned triangulation geometry. A keyframe-gating scheme (`min_parallax_px`) was implemented to only triangulate once tracked features had moved a minimum number of pixels, and the threshold was swept:

`min_parallax_px`	**ATE RMSE**
15	3.678 m
30	3.638 m
50	3.682 m
80	3.620 m

Result was flat — increasing the required baseline by 5× produced no meaningful improvement. This rules out triangulation baseline as the dominant cause.

6.4 Conclusion: the actual root cause

Combining 6.1–6.3: the filter's `predict()` and `update()` steps are each individually correct, but in the full pipeline, landmarks are necessarily triangulated using the **filter's own current pose estimate** (there is no ground truth available at runtime). A slightly-wrong pose produces a slightly-wrong triangulated landmark; that landmark's update then only *partially* corrects the pose that produced it, rather than fully correcting it — and IMU drift (Section 6.1) accumulates between updates faster than these partial corrections can cancel it out.

This is a **known, well-documented limitation of single-pass landmark triangulation** in EKF-based VIO (as opposed to sliding-window approaches like MSCKF, which delay committing to a landmark position until it has been observed with well-constrained uncertainty from multiple poses, and properly propagate pose uncertainty through the triangulation itself). This scaffold's `ekf.py` docstring flagged this simplification up front as a Phase 2 stretch goal; this investigation confirms it is in fact the dominant error source, and rules out the simpler alternative explanations (bad Jacobians, insufficient baseline) that might otherwise be suspected.

7. Known Limitations (carried forward to later phases)

1. **Landmark update uses single-pass triangulation from the filter's own pose estimate**, causing a feedback loop that limits long-run accuracy (Section 6.4). Recommended fix: MSCKF-style structureless update with a sliding window of poses.
2. **IMU mechanization uses first-order Euler integration**, not RK4; fine for the noise levels tested here, but a source of additional integration error at higher dynamics.
3. **Validated only against synthetic data.** Real camera images (rolling shutter, motion blur, variable lighting, non-ideal feature distributions) were not exercised in this phase due to dataset hosting issues (Section 4) and will surface new failure modes.
4. **Monocular scale is only weakly observable** — the recovered scale factor (0.117, Section 5.2) confirms the expected monocular VIO scale-drift behavior; a stereo or metric-scale-aware setup would avoid this.

8. Recommendation

Proceed to **Step 7 (C++ port)** using the current Python implementation as the reference baseline. The numbers in Section 5.2 are the official Phase 1 comparison point. The root-cause analysis in Section 6 should be treated as the starting specification for a Phase 2 "improve estimator accuracy" milestone, rather than something to resolve within Phase 1 — the fix (sliding-window/MSCKF-style update) is a substantial architectural change, not a tuning pass.

9. Reproducing These Results

```
python scripts/generate_synthetic_dataset.py --out data/synthetic_circle --duration 20

python scripts/run_pipeline.py data/synthetic_circle outputs/synth_est.tum

python scripts/evaluate.py data/synthetic_circle outputs/synth_est.tum

python tests/sanity_checks.py
```